

Praticando testes de aplicações



Os **testes unitários** são uma técnica fundamental na garantia da qualidade de software. Eles verificam se **pequenas partes do código** (geralmente funções ou métodos) funcionam corretamente de forma **isolada**. Os testes unitários validam o funcionamento de **uma unidade individual do código**, como uma **função, método ou classe**, sem dependências externas. O objetivo é garantir que cada parte do sistema opere corretamente antes de ser integrada a outras partes.

Objetivo dos Testes Unitários

Os testes unitários têm como principal função validar se cada unidade individual de código — como funções, métodos ou classes — está funcionando conforme o esperado. Entre os principais objetivos, podemos destacar:

- **Garantir a corretude do código**

- Ao testar cada componente isoladamente, é possível verificar se ele está produzindo os resultados corretos em diferentes cenários.

- **Facilitar a detecção de erros cedo no processo de desenvolvimento**

- Testes executados frequentemente permitem identificar falhas logo após a escrita do código, evitando que erros se propaguem para outras partes do sistema.

- **Reduzir custos de manutenção**

- Corrigir um bug identificado precocemente é muito mais barato do que lidar com ele em produção. Testes unitários ajudam a prevenir retrabalho e falhas críticas.

- **Melhorar a qualidade do código**

- Desenvolvedores tendem a escrever códigos mais organizados e modulares quando sabem que eles serão testados de forma automatizada.

- **Apoiar a refatoração segura**

- Mudanças no código são mais confiantes e seguras quando há uma boa cobertura de testes, já que é possível verificar se a funcionalidade original foi preservada.

Características dos Testes Unitários

Para que os testes unitários sejam realmente eficazes, eles devem apresentar as seguintes características:

- **Isolados**

- Cada teste deve se concentrar em uma única unidade de código, sem depender de bancos de dados, serviços externos ou qualquer outro componente que possa introduzir variabilidade nos resultados.

- **Rápidos**

- Testes unitários devem ser executados em milissegundos. Isso permite que rodem frequentemente durante o desenvolvimento, sem comprometer a produtividade.

- **Automatizados**

- A execução dos testes deve ser feita por ferramentas específicas que permitam rodar centenas (ou milhares) de testes de forma automática, como:

- **JUnit** (Java)

- **PyTest** (Python)

- **NUnit** (.NET)

- **Jest** (JavaScript/TypeScript)

- **RSpec** (Ruby)

- **Reprodutíveis**

- Para garantir confiabilidade, os testes devem sempre produzir o mesmo resultado quando executados com os mesmos dados de entrada, independentemente do ambiente.

Os **testes de integração** verificam se diferentes módulos, componentes ou sistemas funcionam corretamente quando conectados. Eles são fundamentais para garantir a **comunicação e interação correta** entre as partes do software.

1. Passos para Criar Testes de Integração

1.1. Planejamento dos Testes

Antes de começar, defina:

- 1. Quais módulos precisam ser testados?**

- 2. Quais interações são mais críticas?**

- 3. Quais são as dependências entre os componentes?**

Exemplo:

No caso de um sistema de e-commerce, algumas integrações críticas incluem:

- Integração entre **pagamento e banco de dados**.

- Integração entre **API do sistema e serviço de e-mails**.

1.2. Escolha do Tipo de Teste de Integração

Big Bang

- Todos os módulos são testados juntos.
- Útil para sistemas pequenos.
- Difícil de depurar se houver muitos erros.

Incremental

- Os módulos são testados **por partes**, conforme são desenvolvidos.
- Pode ser:
 - **Top-down** → Testa primeiro os módulos principais.
 - **Bottom-up** → Testa primeiro os módulos de base.

Testes de Interface/API

- Testa a comunicação entre serviços, geralmente usando chamadas HTTP ou eventos assíncronos.
- **Exemplo:** Um front-end verificando respostas de uma API REST.

Testes de Banco de Dados

- Verifica se o sistema grava e lê corretamente os dados no banco.

Os **testes de sistema** validam o funcionamento do software como um **todo**, garantindo que todos os módulos e componentes trabalhem corretamente juntos. Eles são essenciais para verificar se o sistema atende aos requisitos definidos.

Exemplo:

- Em um **sistema bancário**, o teste verifica **login, transferência de dinheiro, consulta de saldo e geração de extratos** de forma integrada.
- No caso de um **e-commerce**, o teste pode validar **cadastro de usuário, compra, pagamento e envio de e-mails**.

Os **testes de sistema** têm como foco validar o comportamento do sistema como um todo, considerando todos os seus componentes e funcionalidades integrados. Diferente dos testes unitários, que testam partes isoladas, os testes de sistema verificam se todas as partes funcionam corretamente em conjunto, simulando cenários reais de uso.

1. Garantir que todas as funcionalidades do sistema estão funcionando corretamente

- Esse objetivo visa assegurar que o sistema como um todo — com todos os seus módulos já integrados — se comporta conforme o esperado. Aqui são

testadas funcionalidades completas do ponto de vista do usuário, como cadastros, autenticação, geração de relatórios, entre outros.

- Exemplo: Verificar se o fluxo completo de compra em um e-commerce está funcionando, desde a escolha do produto até a confirmação do pagamento.

2. Verificar se o sistema atende aos requisitos especificados

- Os testes de sistema avaliam se o software atende aos requisitos funcionais e não funcionais definidos no início do projeto. Isso garante que o produto corresponde às expectativas do cliente ou usuário final.

- Inclui requisitos como:

- Funcionalidades obrigatórias (ex: login com autenticação em dois fatores)
- Regras de negócio (ex: desconto aplicado apenas para compras acima de R\$ 100,00)
- Requisitos de desempenho (ex: tempo de resposta inferior a 2 segundos)

3. Identificar falhas na interação entre os módulos

- Mesmo que módulos isolados funcionem corretamente, problemas podem surgir na comunicação entre eles. Os testes de sistema ajudam a identificar esses erros de integração que não são visíveis nos testes unitários.

- Exemplos de falhas comuns:

- Dados mal formatados sendo trocados entre APIs
- Dependência incorreta entre módulos
- Problemas de sincronização entre processos

4. Testar a compatibilidade com diferentes dispositivos e plataformas

- Hoje em dia, sistemas precisam funcionar em uma variedade de ambientes — navegadores, sistemas operacionais, tamanhos de tela e dispositivos móveis. Os testes de sistema avaliam essa compatibilidade, garantindo uma boa experiência para todos os usuários.

- Exemplos:

- Verificar se um site funciona corretamente no Chrome, Firefox e Safari
- Garantir que o layout de um aplicativo se adapte bem em diferentes resoluções de tela
- Testar o desempenho em sistemas operacionais como Windows, macOS, Android e iOS

Tipos de Testes de Sistema

Testes Funcionais

Verificam se o sistema realiza corretamente as funções esperadas.

Exemplo: Testar se um usuário pode se cadastrar e fazer login corretamente.

Testes Não Funcionais

Avaliam características como desempenho, segurança e usabilidade.

1. **Teste de Desempenho:** Mede a velocidade e estabilidade do sistema.
2. **Teste de Carga:** Avalia o comportamento sob alto volume de usuários.
3. **Teste de Segurança:** Verifica vulnerabilidades no sistema.
4. **Teste de Compatibilidade:** Confirma se o sistema funciona em diferentes navegadores e dispositivos.

Como Fazer Testes de Sistema?

Passo 1: Definir os Requisitos e Escopo

- Quais funcionalidades serão testadas?
- O que o sistema precisa atender?
- Quais são os principais riscos?

Passo 2: Criar Casos de Teste

Cada **caso de teste** deve conter:

1. **ID** → Identificação única do teste.
2. **Descrição** → O que está sendo validado.
3. **Entradas** → Dados usados no teste.
4. **Passos** → Como executar o teste.
5. **Resultado Esperado** → O que deve acontecer se o teste for bem-sucedido.

Passo 3: Implementação e Execução dos Testes

Os testes podem ser feitos **manualmente** ou **automatizados**.

Exemplo 1: Teste Manual (Checklist de Login)

Passo 1: Abrir o navegador e acessar a página de login.

Passo 2: Inserir um e-mail válido e uma senha correta.

Passo 3: Clicar no botão “Entrar”.

Passo 4: Verificar se o usuário foi direcionado para a tela principal.

Passo 4: Analisar Resultados e Corrigir Defeitos

Registrar falhas encontradas no teste.

Reportar problemas para a equipe de desenvolvimento.

Refazer os testes após as correções.

Os **testes de aceitação** garantem que o sistema atende aos requisitos do cliente e está pronto para ser entregue. Eles validam o software **do ponto de vista do usuário final**, assegurando que todas as funcionalidades atendam às expectativas.

Os **testes de aceitação** representam a etapa final da validação de um sistema

antes da sua entrega ao cliente ou liberação para o ambiente de produção. São executados para garantir que o software está de acordo com as **expectativas do cliente** e **pronto para uso real**. Geralmente, esses testes são realizados pelos próprios clientes, usuários finais ou equipe de QA (Qualidade), com foco em validar a experiência do ponto de vista do negócio.

1. Verificar se o sistema atende aos requisitos do cliente

O principal objetivo é garantir que o sistema entregue cumpra exatamente o que foi acordado com o cliente, tanto em termos de funcionalidades quanto em regras de negócio.

- Os testes se baseiam em **histórias de usuário, casos de uso** ou **critérios de aceitação** definidos no início do projeto.
- Permite que o cliente diga: “Sim, é isso que eu pedi.”

Exemplo: Se um cliente solicitou que um relatório mensal de vendas mostre os totais por categoria, os testes de aceitação irão verificar se essa funcionalidade existe e funciona corretamente.

2. Garantir que todas as funcionalidades principais estão funcionando corretamente

Testes de aceitação validam os fluxos essenciais do sistema — aqueles que impactam diretamente o usuário final e os objetivos do negócio. Isso inclui ações críticas como login, cadastro, finalização de pedido, emissão de notas fiscais, entre outros.

- Foco na **experiência de uso real**: O que acontece se um usuário esquece a senha? Consegue recuperar?
- Avalia se o sistema é utilizável, confiável e intuitivo nas operações mais importantes.

3. Validar se o sistema está pronto para produção

Essa etapa marca a **aprovação final** do sistema. Se os testes forem bem-sucedidos, significa que o software está maduro e pode ser colocado em operação com segurança.

- Garante que não há falhas impeditivas (blockers) ou comportamentos inesperados.
- Envolve testes manuais ou automatizados com dados reais ou cenários muito próximos do uso cotidiano.

Exemplo: Verificar se a integração com o sistema de pagamento de terceiros está funcionando corretamente antes de liberar um e-commerce ao público.

4. Reduzir o risco de falhas após a entrega

Ao simular o uso real do sistema e contar com a validação direta do cliente, os testes de aceitação ajudam a prevenir problemas graves em produção, como:

- Funcionalidades ausentes ou com comportamento incorreto
- Quebras de fluxo que impactam o usuário final
- Rejeição do sistema pelo cliente por falta de aderência aos requisitos

Tipos de Testes de Aceitação

Teste de Aceitação do Usuário (UAT - User Acceptance Testing)

- Realizado pelo **cliente ou usuários finais** para validar se o sistema atende às expectativas.
- Ocorre **antes do sistema entrar em produção**.

Teste de Aceitação de Regressão

- Realiza **novos testes após correções ou melhorias** para garantir que as mudanças não causaram novos erros.

Teste Alfa e Beta

- **Teste Alfa:** Feito internamente pela equipe do cliente antes do lançamento.
- **Teste Beta:** Feito por um grupo de usuários reais para identificar problemas antes da versão final.

Teste de Aceitação Contratual

- Verifica se o sistema atende aos requisitos descritos em um **contrato** ou especificação formal.

Como Fazer Testes de Aceitação?

Passo 1: Definir Critérios de Aceitação

Os critérios de aceitação são **regras** que determinam se uma funcionalidade foi implementada corretamente.

Exemplo: Para um sistema de login:

1. O usuário deve conseguir acessar com um e-mail e senha corretos.
2. Se o login falhar, o sistema deve exibir uma mensagem de erro.
3. Após o login bem-sucedido, o usuário deve ser redirecionado para o dashboard.

Passo 2: Criar Casos de Teste de Aceitação

Cada **caso de teste** deve conter:

1. **ID** → Identificação única do teste.
2. **Descrição** → O que será testado.
3. **Entradas** → Dados que serão usados.
4. **Passos** → Como o teste será executado.
5. **Resultado Esperado** → O que deve acontecer se estiver correto.

Passo 3: Executar os Testes

- Os testes podem ser **manuais** (feitos por usuários reais) ou **automatizados**.
- **Os clientes ou usuários finais executam os testes e reportam os resultados.**

Exemplo 1: Teste Manual de Aceitação

Passo 1: Acessar o site e simular uma compra.

Passo 2: Inserir os dados do cartão e confirmar o pagamento.

Passo 3: Verificar se a transação foi concluída e se um e-mail de confirmação foi recebido.

Passo 4: Registrar e Corrigir Erros

- Se houver falhas, os problemas são documentados e enviados para correção.
- Após as correções, os testes são refeitos para garantir que o erro foi resolvido.

GitHub da Disciplina

Confira os códigos e scripts usados ao longo da disciplina no repositório dela no GitHub: <https://github.com/FaculdadeDescomplica/Pratica-Integradora-com-Metodos-Ageis>

Conteúdo Bônus

Uma série que mostra de uma forma bem-humorada os bastidores do desenvolvimento de software com bugs inesperados, deploys desastrosos e testes malfeitos é a série Silicon Valley de 2014, nela podemos ver como é importante ter testes bem executados e de boa confiabilidade.

Referências Bibliográficas

BURNSTEIN, Ilene. Practical Software Testing: A Process-Oriented Approach. New York: Springer, 2003.

GLENFORD, Myers. The Art of Software Testing. Hoboken: John Wiley & Sons, 2011.

LIGOWSKI, Tomek. Software Testing: Principles and Practices. Londres: Pearson, 2008.

MESZAROS, Gerard. xUnit Test Patterns: Refactoring Test Code. Boston: Addison-Wesley, 2007.

Diferentes tipos de testes de software. Sten Pittet. Disponível em <https://www.atlassian.com/br/continuous-delivery/software-testing/types-of-software-testing>. Acesso em 11 de abril de 2025.

O que é teste de software? IBM. Disponível em <https://www.ibm.com/br-pt/topics/software-testing>. Acesso em 11 de abril de 2025.

Técnicas e fundamentos de Testes de Software. DevMedia. Disponível em <https://www.devmedia.com.br/guia/tecnicas-e-fundamentos-de-testes-de-software/34403>. Acesso em 11 de abril de 2025.

16 Software Testing Best Practices to Follow Now. Higher School of Networks, maio de 2024. Disponível em <https://esr.rnp.br/desenvolvimento-de-sistemas/melhores-praticas-em-testes-de-software/>. Acesso em 11 de abril de 2025.

Atividade Prática 8 – Praticando testes de aplicações

Título da Prática: Praticando testes de aplicações

Objetivos: Exercitar os conceitos dos tipos de testes

Materiais, Métodos e Ferramentas: Para realizar esta prática vamos ler atentamente o conteúdo da unidade

Atividade Prática

Uma equipe de desenvolvimento está construindo um sistema de gerenciamento de biblioteca que possui várias funcionalidades, como cadastro de livros, empréstimos, devoluções e multas. Para garantir a qualidade do software e prevenir falhas, o time decide adotar diversas práticas de teste: testes unitários, testes de integração, testes de sistema e testes de aceitação. Como membro da equipe, você foi encarregado de explicar e exemplificar essas práticas para novos integrantes.

1. Explique com suas palavras o conceito de **testes unitários** e cite um exemplo aplicado ao sistema de gerenciamento de biblioteca.
2. Descreva o que são **testes de integração** e como eles podem ser realizados no contexto do sistema.
3. Defina o que são **testes de sistema** e exemplifique uma situação em que esse tipo de teste seria aplicado no sistema da biblioteca.
4. Explique o conceito de **testes de aceitação** e dê um exemplo de um teste desse tipo no sistema da biblioteca.

Gabarito Atividade Prática

- 1.** Testes unitários são testes que verificam o funcionamento correto de uma unidade isolada do código, como uma função ou um método. No sistema de gerenciamento de biblioteca, um exemplo seria testar uma função que calcula a multa de um livro atrasado, verificando se ela retorna o valor correto com base na quantidade de dias de atraso.
- 2.** Testes de integração verificam se diferentes módulos ou componentes do sistema funcionam corretamente quando combinados. No sistema de biblioteca, um exemplo seria testar a integração entre o módulo de empréstimos e o módulo de atualização de estoque, para garantir que ao emprestar um livro, a quantidade disponível no estoque seja reduzida automaticamente.
- 3.** Testes de sistema validam o comportamento do sistema como um todo, verificando se ele atende aos requisitos especificados. No caso da biblioteca, um exemplo seria simular o processo completo de um usuário que faz o cadastro, realiza um empréstimo e depois devolve o livro, garantindo que todas essas funcionalidades funcionam corretamente em conjunto.
- 4.** Testes de aceitação são realizados para verificar se o sistema atende aos critérios de aceitação definidos pelo cliente ou usuário final. No sistema de biblioteca, um exemplo seria validar, junto ao bibliotecário, se o sistema permite registrar um empréstimo em menos de dois minutos, conforme requisito estipulado pelo cliente.